Visitor Design Pattern (Behavioral)

Separate an **operation** from the **object structure** it works on, so new operations can be added without ever modifying the objects being operated on.

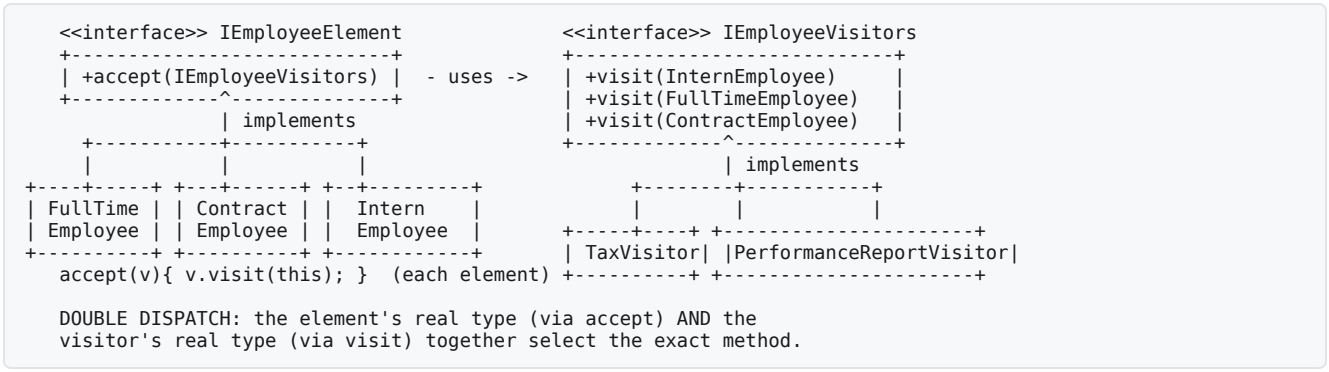
Intent

The Visitor pattern lets you define a new operation over a hierarchy of object types **without changing the classes of those objects**. Instead of adding a method to every element class each time you need a new operation, you put each operation into its own **visitor** class. The elements expose a single `accept(visitor)` method and hand themselves back to the visitor, which carries the actual behaviour.

Here the object structure is a set of **employee types** — full-time, contract, and intern — and the operations are things you might *do* to an employee: compute a **tax report** or a **performance report**. Adding a brand-new operation (say, a bonus calculator) means writing exactly one new visitor class and touching **none** of the employee classes.

This is the **canonical GoF Visitor**: an element interface with a single `accept` method, one concrete element per type, a visitor interface with one overloaded `visit(...)` per element type, and one concrete visitor per operation. The mechanism that makes it work is **double dispatch**, explained in full below.

UML class diagram (ASCII)



The players

```
elements/IEmployeeElement                       the element contract: accept(visitor)
elements/concrete/FullTimeEmployee              concrete elements — each accept() body is
                 /ContractEmployee               just visitor.visit(this)
                 /InternEmployee

visitors/IEmployeeVisitors                       the visitor contract: one visit(...) overload
                                                 per concrete element type
visitors/concrete/TaxVisitor                     operation #1 — tax report per employee type
                 /PerformanceReportVisitor       operation #2 — performance report per type

VisitorDesignPattern                             the demo — runs every visitor over every element
```

Two separate hierarchies that meet in the middle:

- **Elements** = the data (employee types), under `elements/` and `elements/concrete/`.
- **Visitors** = the operations (reports), under `visitors/` and `visitors/concrete/`.

Both concrete visitors — `TaxVisitor` **and** `PerformanceReportVisitor` — are declared `public`.

Code walkthrough — every line explained

`IEmployeeElement.java` — the element contract

```
package com.design.patterns.visitor.elements;

import com.design.patterns.visitor.visitors.IEmployeeVisitors;

public interface IEmployeeElement {
    void accept(IEmployeeVisitors visitor);
}
```

- `package com.design.patterns.visitor.elements;` — Places the element contract in the `elements` package, keeping the "data" side of the pattern separate from the "operation" (`visitors`) side.
- `import com.design.patterns.visitor.visitors.IEmployeeVisitors;` — Brings the visitor interface into scope so `accept` can name it as a parameter type. This is the single point where the element side references the visitor side.
- `public interface IEmployeeElement {` — Declares the element interface every employee type implements. `public` so both the concrete elements and the demo (in other packages) can use it.
- `void accept(IEmployeeVisitors visitor);` — The **one** method of the pattern's element side. It means "a visitor has arrived — let it operate on me." Crucially, the element does **not** perform the operation itself; it only **admits** the visitor. What actually happens is decided later by the visitor's real type combined with the element's real type (double dispatch). Returning `void` keeps the demo simple; a real system could return a result or accumulate it in the visitor.

The concrete elements — `FullTimeEmployee` , `ContractEmployee` , `InternEmployee`

All three are identical in shape. `FullTimeEmployee.java` :

```
package com.design.patterns.visitor.elements.concrete;

import com.design.patterns.visitor.elements.IEmployeeElement;
import com.design.patterns.visitor.visitors.IEmployeeVisitors;

public class FullTimeEmployee implements IEmployeeElement {

    @Override
    public void accept(IEmployeeVisitors visitor) {
        visitor.visit(this);
    }
}
```

- `package com.design.patterns.visitor.elements.concrete;` — The concrete elements live in the `concrete` sub-package, separating the interface from its implementations.
- `import ... IEmployeeElement;` — Imports the interface this class implements.
- `import ... IEmployeeVisitors;` — Imports the visitor type used as the `accept` parameter.
- `public class FullTimeEmployee implements IEmployeeElement {` — A concrete employee type. It carries no fields here so the code spotlights the dispatch mechanism rather than data.
- `@Override public void accept(IEmployeeVisitors visitor) {` — Implements the element contract. The compiler-checked `@Override` guarantees the signature matches `IEmployeeElement.accept` .
- `visitor.visit(this);` — **The linchpin of the whole pattern.** The word `this` matters: inside `FullTimeEmployee.accept` , `this` has the *static type* `FullTimeEmployee` , so the compiler selects the `visit(FullTimeEmployee)` overload. Inside `InternEmployee.accept` the same line resolves to `visit(InternEmployee)` , and inside `ContractEmployee.accept` to `visit(ContractEmployee)` . Each element "knows its own type" and uses it to route to the correct overload on the visitor. The three classes are textually identical but **not redundant** — the type of `this` differs, so each sends control to a different visitor method. You cannot collapse them into one shared method, because the concrete type of `this` is exactly the information being exploited.

`ContractEmployee` and `InternEmployee` are the same except for the class name (and therefore the type of `this`).

`IEmployeeVisitors.java` — the visitor contract

```
package com.design.patterns.visitor.visitors;

import com.design.patterns.visitor.elements.concrete.ContractEmployee;
import com.design.patterns.visitor.elements.concrete.FullTimeEmployee;
import com.design.patterns.visitor.elements.concrete.InternEmployee;

public interface IEmployeeVisitors {

    void visit(InternEmployee internEmployee);

    void visit(FullTimeEmployee fullTimeEmployee);

    void visit(ContractEmployee contractEmployee);
}
```

- `package com.design.patterns.visitor.visitors;` — Places the visitor contract in the `visitors` package.

- The three `import` lines pull in the concrete element types from `elements.concrete` so the overloads can name them. The visitor side must know every concrete element type — that coupling is inherent to the pattern.
- `public interface IEmployeeVisitors {` — The visitor contract: "I promise to know what to do with every kind of employee."
- `void visit(InternEmployee internEmployee);` / `void visit(FullTimeEmployee fullTimeEmployee);` / `void visit(ContractEmployee contractEmployee);` — **One overloaded `visit(...)` method per concrete element type.** This is the second half of double dispatch: the `visitor.visit(this)` call in each element resolves to exactly one of these overloads at compile time based on the static type of `this`. Listing every element type here is also the pattern's central trade-off in plain sight: add a new employee type and you must add a `visit(...)` overload here — and then implement it in **every** visitor.

`TaxVisitor.java` — concrete operation #1

```
package com.design.patterns.visitor.visitors.concrete;

import com.design.patterns.visitor.elements.concrete.ContractEmployee;
import com.design.patterns.visitor.elements.concrete.FullTimeEmployee;
import com.design.patterns.visitor.elements.concrete.InternEmployee;
import com.design.patterns.visitor.visitors.IEmployeeVisitors;

public class TaxVisitor implements IEmployeeVisitors {

    @Override
    public void visit(FullTimeEmployee employee) {
        System.out.println("Generating tax report for full-time employee.");
    }

    @Override
    public void visit(ContractEmployee employee) {
        System.out.println("Generating tax report for contract employee.");
    }

    @Override
    public void visit(InternEmployee internEmployee) {
        System.out.println("Generating tax report for intern employee.");
    }
}
```

- `package com.design.patterns.visitor.visitors.concrete;` — Concrete visitors live in the `concrete` sub-package, mirroring the element side's layout.
- The four `import` lines bring in the three concrete element types (needed for the overload parameters) and the `IEmployeeVisitors` interface this class implements.
- `public class TaxVisitor implements IEmployeeVisitors {` — A concrete visitor. Declared `public` so it can be constructed from the demo in the parent package. Implementing `IEmployeeVisitors` forces it (by the compiler) to provide all three `visit` overloads — you cannot forget one.
- Each `@Override public void visit(...) { System.out.println("Generating tax report for ... employee."); }` — Supplies the **tax** behaviour for one specific employee type. The payoff of the pattern is visible here: everything about "computing tax" for all three employee types is bundled into this one cohesive class, instead of being scattered as a `computeTax()` method across `FullTimeEmployee`, `ContractEmployee`, and `InternEmployee`. The `System.out.println(...)` stands in for real logic (in a real system it would read salary/hours from the element and compute a number). The string literals are the observable proof of which overload ran.

`PerformanceReportVisitor.java` — concrete operation #2

Structurally identical to `TaxVisitor`, but each `visit` prints a **performance** message instead of a tax one:

```
package com.design.patterns.visitor.visitors.concrete;

import com.design.patterns.visitor.elements.concrete.ContractEmployee;
import com.design.patterns.visitor.elements.concrete.FullTimeEmployee;
import com.design.patterns.visitor.elements.concrete.InternEmployee;
import com.design.patterns.visitor.visitors.IEmployeeVisitors;

public class PerformanceReportVisitor implements IEmployeeVisitors {

    @Override
    public void visit(FullTimeEmployee employee) {
        System.out.println("Generating performance report for full-time employee.");
    }

    @Override
    public void visit(ContractEmployee employee) {
        System.out.println("Generating performance report for contract employee.");
    }

    @Override
    public void visit(InternEmployee internEmployee) {
        System.out.println("Generating performance report for intern employee.");
```

```
    }
}
```

- Everything said about `TaxVisitor` applies. This class is also `public`. It demonstrates the pattern's headline benefit directly: a **second operation** over the *same* element hierarchy was added purely by writing a new class — not a single employee class changed. `TaxVisitor` and `PerformanceReportVisitor` are two independent, cohesive operations sitting side by side.

`VisitorDesignPattern.java` — the demo / driver

```
package com.design.patterns.visitor;

import java.util.List;

import com.design.patterns.visitor.elements.IEmployeeElement;
import com.design.patterns.visitor.elements.concrete.ContractEmployee;
import com.design.patterns.visitor.elements.concrete.FullTimeEmployee;
import com.design.patterns.visitor.elements.concrete.InternEmployee;
import com.design.patterns.visitor.visitors.IEmployeeVisitors;
import com.design.patterns.visitor.visitors.concrete.PerformanceReportVisitor;
import com.design.patterns.visitor.visitors.concrete.TaxVisitor;

public class VisitorDesignPattern {

    public static void main(String[] args) {
        System.out.println("Visitor Design Pattern");

        List<IEmployeeElement> employees = List.of(new InternEmployee(), new FullTimeEmployee(),
                new ContractEmployee());

        for (IEmployeeVisitors visitor : List.of(new TaxVisitor(), new PerformanceReportVisitor())) {
            employees.forEach(employee -> employee.accept(visitor));
        }
    }
}
```

- `package com.design.patterns.visitor;` — The driver sits in the root package, one level above `elements` and `visitors`, keeping the demo separate from the pattern's building blocks.
- `import java.util.List;` — Brings in `List` for the two immutable collections built below.
- The remaining imports pull in the element interface, all three concrete elements, the visitor interface, and both concrete visitors — everything the demo constructs.
- `public class VisitorDesignPattern {` — The client/driver class. `public` so the JVM can resolve `main` by name from the command line.
- `public static void main(String[] args) {` — Standard Java entry point. `static` so no instance is needed; `String[] args` receives command-line arguments (unused here).
- `System.out.println("Visitor Design Pattern");` — Prints the header line so the output has a clear title before the six report lines.
- `List<IEmployeeElement> employees = List.of(new InternEmployee(), new FullTimeEmployee(), new ContractEmployee());` — Builds an immutable list of one of each employee type, **referenced through the `IEmployeeElement` interface**. Referencing through the interface is deliberate: the demo never uses the concrete types after construction, which is what forces double dispatch to do the type-recovery work at run time.
- `for (IEmployeeVisitors visitor : List.of(new TaxVisitor(), new PerformanceReportVisitor())) {` — Iterates over an immutable list of the two operations, each referenced through the `IEmployeeVisitors` interface. Adding a third operation would mean adding one more element to this list — and writing one new visitor class, nothing else.
- `employees.forEach(employee -> employee.accept(visitor));` — For the current visitor, applies it to **every** employee via `accept`. This is the **first dispatch**: `accept` is a virtual call on the element, so the JVM picks the right override (`InternEmployee.accept`, etc.) based on the element's real type. Inside that override, `visitor.visit(this)` performs the **second dispatch**. The nested loops run 2 visitors × 3 employees = **6** `accept` calls, producing 6 report lines (plus the 1 header line printed earlier = 7 lines total).

Why these design decisions

Why not just put `computeTax()` / `generateReport()` methods on the employees?

You could — but then **every operation is smeared across every element class**, and each new operation forces you to edit *all* the employee classes. Worse, operations that don't really belong to "being an employee" (tax rules, report formatting) pollute the data classes. Visitor **pulls each operation into its own class**, so `TaxVisitor` holds all tax logic for all employee types in one cohesive place, and the employee classes stay clean and stable.

Why the `accept` / `visit(this)` indirection at all?

Purely to achieve double dispatch (see the execution trace below). It is the only way in a single-dispatch language like Java to let *two* run-time types jointly decide which method runs. The `accept` method exists solely to recover the

element's concrete type and forward it — via the static type of `this` — to the correctly-typed `visit` overload.

Why one `visit(...)` overload per element type?

So each visitor is **forced by the compiler** to handle every kind of element. If you add `visit(FreelancerEmployee)` to the interface, every existing visitor stops compiling until it provides that behaviour — a *feature*, because you cannot accidentally forget to define how tax works for a new employee type.

The fundamental trade-off (know this cold)

Visitor makes one axis easy and the other hard:

- **Adding a new operation is trivial** — write one new visitor class (e.g. `BonusVisitor`), change nothing else. This is exactly why the demo could add `PerformanceReportVisitor` alongside `TaxVisitor` without editing any element.
- **Adding a new element type is expensive** — a new `IEmployeeElement` means adding a `visit(...)` method to the visitor interface *and* implementing it in **every** existing visitor.

Pick Visitor when the **element hierarchy is stable** and you keep needing **new operations** over it. If your element types churn more than your operations, prefer methods on the elements instead.

Execution flow (step-by-step trace of the double dispatch)

Take the very first iteration: the `TaxVisitor` visiting the `InternEmployee`.

```
main
 │
 ├── employees = [ InternEmployee, FullTimeEmployee, ContractEmployee ]   (typed as IEmployeeElement)
 ├── visitors  = [ TaxVisitor, PerformanceReportVisitor ]                 (typed as IEmployeeVisitors)
 │
 └── employee.accept( taxVisitor )       employee's static type is IEmployeeElement
          │   ── 1st DISPATCH: virtual call selects InternEmployee.accept   (element's REAL type)
          ▼
     InternEmployee.accept(visitor) { visitor.visit(this); }
          │   ── here `this` is statically an InternEmployee
          │      → the compiler binds the call to the visit(InternEmployee) overload
          │   ── 2nd DISPATCH: virtual call selects TaxVisitor.visit         (visitor's REAL type)
          ▼
     TaxVisitor.visit(InternEmployee)
          └── prints "Generating tax report for intern employee."
```

Why two dispatches are needed. Java method calls are **single dispatch**: the method that runs is chosen by the run-time type of exactly **one** object — the receiver before the dot. Overloads like `visit(FullTimeEmployee)` vs `visit(InternEmployee)` are resolved by the **compiler** using the **static** type of the argument. So writing `taxVisitor.visit(employee)` directly would fail — the compiler only knows `employee` as `IEmployeeElement`, and there is no `visit(IEmployeeElement)` overload. Visitor recovers the missing type information in two steps:

1. **First dispatch — `element.accept(visitor)`** — a normal virtual call on the *element*. The JVM picks the override matching the element's real type, so control lands inside a method where the concrete type is statically known.
2. **Second dispatch — `visitor.visit(this)`** — inside that override, `this` has the concrete static type, so the compiler binds the correct `visit(...)` overload; the virtual call then selects the concrete visitor at run time.

The **pair** of real types — element and visitor — selects the exact method (`TaxVisitor.visit(InternEmployee)`). That is double dispatch, and it is the entire mechanical reason the pattern is shaped this way.

The demo repeats this for all 2 × 3 = 6 combinations, so every employee type is reported on by every visitor.

Expected output

Running `VisitorDesignPattern.main` prints exactly these 7 lines (1 header + 3 employees × 2 visitors):

```
Visitor Design Pattern
Generating tax report for intern employee.
Generating tax report for full-time employee.
Generating tax report for contract employee.
Generating performance report for intern employee.
Generating performance report for full-time employee.
Generating performance report for contract employee.
```

The visitors are applied in list order (`TaxVisitor` then `PerformanceReportVisitor`), and within each visitor the employees are visited in list order (intern, full-time, contract) — which is exactly the order of the lines above.

How to run

```
# From the module root directory
JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64 mvn -o clean compile

java -cp target/classes com.design.patterns.visitor.VisitorDesignPattern
```

The `-o` (offline) flag works once the parent reactor and dependencies are already in your local Maven cache; drop it on a first build. This module is plain Java with a `main` method — no Spring container is involved, so the JVM prints the seven lines and exits immediately.

Relationship to the other Behavioral patterns

- **Visitor (this module)** — add **operations** over a fixed set of element types without changing them; relies on double dispatch.
- **Strategy** — swap **one** algorithm on a single object; no traversal of a type hierarchy.
- **Template Method** — fix an algorithm's skeleton, vary steps via inheritance.
- **Chain of Responsibility** — pass a request along handlers until one handles it.

Reach for Visitor when you have a **stable hierarchy of types** and an **open-ended, growing set of operations** you want to run over them — and you'd rather add operations as new classes than keep editing the types.